

Exact populations

Configurations, datasets and addressable mathematics

Finite exact configuration spaces, reversible global ranks, sampling without replacement, original-coordinate lookup and reproducible mathematical datasets.

768

related arithmetic tasks

156

sourced unit domains

512

sourced index records

Standard-library application layer over existing complete arithmetic. Sampling and rank transport are independently tested; no new Lean proof or industrial performance claim is made.

Source, corpus receipts and reproduction commands accompany the repository snapshot.

The new computational object

PerfectPower now exposes finite supported solution spaces as addressable populations. Compile a polynomial/modular domain or a supported integer curve, then count its objects, select an object by global rank, recover a rank from its identity, sample with or without replacement, divide work into rank shards, and export a reproducible dataset. The output is an actual configuration or original integer point. This development implements the common foundation for configuration generation, mathematical datasets, embedded query services and compact mathematical catalogues discussed in the direct-use roadmap.

The implementation uses only the Python standard library and the repository's existing arithmetic modules. It introduces no new mathematical priority claim. A general exact population is not a general solver for integer polynomial systems. The new capability is a reusable interface over already complete finite domains, disjoint explicit curve charts and supported complete finite nonlinear-image lists.

Configuration identity and projected values

A domain population consists of admissible integer parameters n . Each parameter carries named integer polynomial fields, such as `square_side= $n^3$` , `cube_side= $n^2$` and `elements= $n^6$` . Parameter identity is retained even if two parameters have identical projected fields. For example, $n=-2$ and $n=2$ both produce `square=4`; they are two configurations if parameter is part of identity. Uniform sampling over those configurations is not uniform sampling over the three distinct square values produced by n in $[-2,2]$. The interface makes this distinction explicit rather than silently deduplicating values.

A curve population consists of original integer points (x,y) . The existing chart compiler gives each original point exactly one explicit chart/parameter owner, including ownership of zero. Consequently counting chart parameters counts original points once. If a nonlinear image closes into a complete finite point list, the population instead uses the sorted original list. An unresolved image does not receive a population count or sampler.

The specification is copied and normalized at construction. A SHA-256 population identifier binds the normalized specification and schema version; it is an identifier, not a formal proof or a signature. Evidence and specifications returned to callers are copies. Public cardinality and identifier properties cannot be reassigned. A newly restricted population receives its own specification and identifier.

A reversible address space

For explicit charts, order first by the existing chart order and then by increasing admissible parameter. This is chart-major ordering, not increasing x or lexicographic point ordering. Let C_i be chart i 's cardinality and $B_i = \sum_{j < i} C_j$ its rank offset. A point at local parameter rank r in chart i has global rank $B_i + r$. Empty charts retain their mathematical branch number and contribute no ranks. Repeated cumulative offsets are handled without assigning a rank to an empty branch.

Within each interval/residue cell, exact counts use floor differences. Selection uses those counts and binary search, with the finite domain's true lower endpoint as its start. Negative parameters therefore participate in the same ordering. Rank recovery counts admissible parameters before the supplied parameter and adds the chart offset. It re-evaluates original field values rather than trusting the record's rank field.

The locate operation accepts a domain parameter or an original curve point. Curve location solves a complete univariate inverse coordinate fibre, checks every integer root against the chart domain, and checks the second coordinate. Constant zero charts are handled directly. This inverse discovery uses the existing bounded integer-image cache. Ordinary record rank recovery needs no inverse-root discovery. Selection and dataset generation use the already compiled chart domains.

The page operation materializes a bounded rank window directly, without traversing earlier objects. The next operation returns the next configuration in domain-parameter order. Rank shards use boundaries $\text{floor}(N*i/s)$ and $\text{floor}(N*(i+1)/s)$; the resulting half-open intervals partition every rank exactly once and have sizes differing by at most one. Shards support external workers without materializing the full population.

Exact sampling without a population-sized allocation

Sampling with replacement chooses a rank uniformly from 0 through $N-1$ for each requested record. Without replacement, a sparse partial Fisher-Yates procedure maintains only a dictionary of moved rank positions. At step i , draw a position from the remaining $N-i$ positions, return its mapped rank, replace that position by the final remaining position, and remove the final position from the sparse dictionary.

Under independent uniform draws, every ordered sample of k distinct ranks has probability $1/(N*(N-1)*\dots*(N-k+1))$. A reversible rank map transports this measure to the population objects. The procedure stores $O(k)$ rank entries, not $O(N)$. Its complete runtime also includes rank selection, integer arithmetic and field evaluation, so no claim that all work is $O(k)$ is made.

Python's seeded Random generator supplies reproducible pseudorandom draws. This is not a cryptographic generator. Dataset metadata retains seed, replacement mode, sampler version and Python version. Reproduction is scoped to the recorded runtime behavior; an unqualified cross-version random-stream guarantee is not claimed. An exhaustive four-object, three-draw test enumerates all 24 possible draw paths and obtains every ordered triple exactly once. That test substantiates the sampling construction; it does not claim a formal Lean proof of the Python program.

Direct layout generation

The saved layout population uses $1 \leq n \leq 10^{30}$ and n modulo eight in $\{0,3\}$. Its fields are $\text{square_side}=n^3$, $\text{cube_side}=n^2$ and $\text{elements}=n^6$. Every emitted configuration has $\text{square_side}^2=\text{cube_side}^3=\text{elements}$. The exact population cardinality is 250000000000000000000000000000, and the runner creates 128 distinct sampled configurations.

The example also minimizes $(n - (10^{30} - 5))^2$ over that admissible domain. The exact optimum is zero at $n = 999999999999999999999999999995$. This is an arithmetic layout-design example, not a hardware throughput experiment or a physical performance model. A real application can attach capacity, memory or experiment requirements to the generated dimensions. The domain optimizer accepts a polynomial in the parameter; it does not infer an arbitrary objective written in the projected field names.

A mathematical dataset with an enormous population

[illegible]

Those records supply 768 related tasks: selecting a ranked original point, evaluating a specified chart parameter, and counting a restricted family whose parameter ceiling is taken from the sample. The count answers use the independent closed formula $2^t + 1$. These are generated arithmetic tasks, not an evaluation result for any AI system. They illustrate how one mathematical population can support several linked questions while preserving their common structure.

The finite nonlinear-image example $(x^2-1)^2=y^3$ has five original points: $(-3,4)$, $(-1,0)$, $(0,1)$, $(1,0)$ and $(3,4)$. The previously documented three tied minimizers are an optimizer subset, not the entire curve. The new tests retain all five points and recover their ranks in the complete finite list.

Sourced combinatorial domains

The existing data/gamma_bober52.json contains the 52 sporadic factorial-ratio parameter rows sourced from Jonathan W. Bober's Table 2. The development runner derives their original polynomial-coefficient recurrences $Q(n) \cdot A(n+1) = P(n) \cdot A(n)$. For each family and prime in $\{17, 31, 47\}$, it compiles indices $0 \leq n \leq 10^{100}$ where both $P(n)$ and $Q(n)$ are nonzero modulo that prime.

There are 156 such domains, of which 128 are nonempty, producing 512 sampled index records. An independent reference evaluates the original affine factorial-increment factors modulo each prime and counts the allowed residues using floor formulas. Every compiled count agrees with that reference, and every emitted index belongs to an allowed

residue. Source-file and original-source provenance accompany the domain specifications.

These are exact recurrence-coefficient unit domains. They do not classify the integer factorial ratio as a perfect power. The mathematical parameters are independently sourced; the dataset requests and selected bounds are constructed here. This example shows a useful source-to-population workflow without relabelling generated queries as industrial workloads.

Public Python interface

```
from perfectpower.populations import ExactPopulation

spec = {
    "kind": "domain",
    "predicate": {"op": "and", "args": [
        {"poly": [-1, 1], "relation": ">="},
        {"poly": [-1000, 1], "relation": "<="},
        {"poly": [0, 1], "modulus": 8,
         "relation": "=", "value": 3}
    ]},
    "fields": {"square_side": [0, 0, 0, 1],
               "cube_side": [0, 0, 1]}
}
population = ExactPopulation(spec)
record = population.select(5)
assert population.rank(record) == 5
assert population.locate(parameter=record["parameter"]) == 5
sample = population.sample(20, seed=20261005)
next_configuration = population.next(100)
population.export("configurations.jsonl", size=20, seed=20261005)
```

A curve specification uses kind=curve, ascending-coefficient left and right polynomials, and a structured original-coordinate predicate using expr atoms in x,y. Count, select, rank, locate, sample, page, partition, restrict, optimize and export are exposed by the same population object. For curves, optimize accepts an expression in the original x,y coordinates. Domain optimization accepts ascending parameter-polynomial coefficients. Existing optimization engines create their own query-specific domain and difference evidence; repeated rank and sample calls do not recompile the source domain.

Command-line workflows

```
PYTHONPATH=python python python/develop_populations.py
python -m perfectpower population \
  --spec receipts/populations/compatible_layouts.spec.json
python -m perfectpower population \
  --spec receipts/populations/coefficient_curve.spec.json \
  --sample 100 --seed 42 --output points.jsonl
python -m perfectpower population \
  --spec receipts/populations/compatible_layouts.spec.json \
  --page '[1000000,10]'
python -m perfectpower population \
  --spec receipts/populations/coefficient_curve.spec.json \
  --locate '{"x":-144,"y":24}'
```

The first JSONL line contains metadata; subsequent lines contain records with population identifier, global rank, chart, parameter and original values. File publication uses a temporary sibling file, flush/fsync and atomic replacement after all records have been constructed. A failed sample or value budget leaves an existing destination intact. JSON numbers are exact integers; consumers must use arbitrary-precision integer parsing rather than converting large identifiers, ranks or coordinates to IEEE floating-point numbers.

Resource limits and proof scope

The population wrapper defaults to 100,000 materialized rows, 100,000 root nodes, residue period 65,536, one million domain work units and 4,096 output bits. Existing polynomial degree and coefficient budgets also apply. An infinite source needs finite restrictions before construction. Complete nonlinear-image point lists must fit the row budget. Explicit structured populations need not fit that row budget in cardinality; only requested materialization is bounded.

The source mathematics inherits its existing exact Python evidence and relevant Lean theorem provenance. This release adds no Lean declaration and does not make the Python population compiler, sampler or exporter kernel-verified. Sampling uniformity is a mathematical argument about uniform draws and reversible ranks. The program is tested against independent populations and exhaustively enumerated small sampling paths. General integer solving, arbitrary nonlinear parameter images, physical performance optimization and uniform sampling over deduplicated projected values remain outside the interface.

Tests and reproduction

The focused suite contains 13 test methods covering 60 independently scanned random domains, signed curve points against exhaustive boxes, negative and disconnected parameter domains, original-point location, empty charts, finite nonlinear images, huge ranks, balanced sharding, exhaustive sampling paths, repeated field values, seeded reproduction, replacement, restrictions, tied optima, budgets, copied evidence, atomic export and the public command line. Run the focused command below or make test for the four repository Python suites.

```
PYTHONPATH=python python -m unittest discover \
  -s python/tests -p test_populations.py -v
make test
```

The runner rebuilds its receipts and datasets from repository sources. Its output includes source hashes, population specifications, independent reference counts, selected original values and generated tasks. A complete handoff archive accompanies this monograph. The direct-use roadmap records which application fronts are implemented by this common layer and which require additional domain-specific work.